TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN GIỮA KỲ

504070 — Service-Oriented Architecture
(Kiến trúc hướng dịch vụ)

ĐỀ TÀI: PHÂN HỆ ĐÓNG HỌC PHÍ TRÊN ỨNG DỤNG IBANKING THEO KIẾN TRÚC MICROSERVICES

Giảng viên hướng dẫn: Dương Hữu Phúc

Sinh viên thực hiện: Nguyễn Hoàng Thanh Minh

Mã số sinh viên: 519H0194

Mã nhóm:

Học kỳ: HK1 / 2026–2027

TP. HỒ CHÍ MINH, 2026

MỤC LỤC

LỜI MỞ ĐẦU
1. Phân tích nghiệp vụ và dữ liệu
    1.1. Tác nhân (Actor)
    1.2. Use Case Diagram
    1.3. Sơ đồ trạng thái của giao dịch
    1.4. Entity Relationship Diagram
2. Thiết kế kiến trúc Microservices
    2.1. Microservices Architecture Diagram
    2.2. Phân rã service và trách nhiệm
    2.3. Giao tiếp giữa các service
    2.4. Sequence Diagram — luồng thanh toán đầy đủ
    2.5. Giải thích các quyết định kiến trúc
    2.6. Deployment view
3. Thiết kế REST API
    3.1. Quy ước chung
    3.2. API công khai (qua API Gateway)
    3.3. API nội bộ (service-to-service)
    3.4. Validation và error handling
4. Xử lý Transaction và Concurrency
    4.1. Hai tầng nhất quán
    4.2. Transaction cục bộ
    4.3. Saga và bù trừ
    4.4. Tình huống 5.1 — Nhiều giao dịch trên cùng một tài khoản
    4.5. Tình huống 5.2 — Nhiều tài khoản thanh toán cùng một khoản học phí
    4.6. Bảng tổng hợp cơ chế
    4.7. Giao dịch treo: job đối soát tự động
    4.8. Cách chứng minh khi demo
5. Hướng dẫn sử dụng và kịch bản demo
    5.1. Tài khoản và dữ liệu mẫu
    5.2. Hướng dẫn sử dụng hệ thống
    5.3. Kịch bản demo trước hội đồng (≈ 10 phút)
    5.4. Câu hỏi thường gặp khi bảo vệ
6. Kiểm thử và minh chứng thực nghiệm
    6.1. Môi trường và kết quả
    6.2. Truy vết phân tán
    6.3. Giao diện và luồng sử dụng
    6.4. Phạm vi chứng minh
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN
    Kết quả đạt được
    Hướng phát triển
TÀI LIỆU THAM KHẢO

LỜI MỞ ĐẦU

Trong bối cảnh chuyển đổi số giáo dục đại học và tài chính số phát triển nhanh chóng, việc số hóa các dịch vụ tiện ích sinh viên ngày càng đóng vai trò quan trọng. Đồ án môn học **Kiến trúc hướng dịch vụ (504070)** tập trung hiện thực **Phân hệ đóng học phí trên ứng dụng iBanking**, mô phỏng nghiệp vụ tài chính thực tế giữa Ngân hàng và Nhà trường.

Hệ thống được thiết kế hoàn toàn theo mô hình **Microservices Architecture**, phân rã thành các dịch vụ nghiệp vụ chuyên biệt: `auth-service`, `account-service`, `tuition-service`, `payment-service`, `otp-service`, `notification-service`, cùng với thành phần điều phối `api-gateway` và giao diện người dùng `web-client`. Mỗi dịch vụ quản lý cơ sở dữ liệu riêng biệt (Database per Service), trao đổi qua REST API và đảm bảo tính nhất quán dữ liệu phân tán bằng mẫu hình **Saga Orchestration** với cơ chế bù trừ giao dịch (Compensating Transaction) và Idempotency.

Báo cáo này tổng hợp đầy đủ thiết kế phân tích nghiệp vụ, kiến trúc kỹ thuật, quy chuẩn API, giải pháp giao dịch phân tán và hướng dẫn vận hành hệ thống.

1. Phân tích nghiệp vụ và dữ liệu

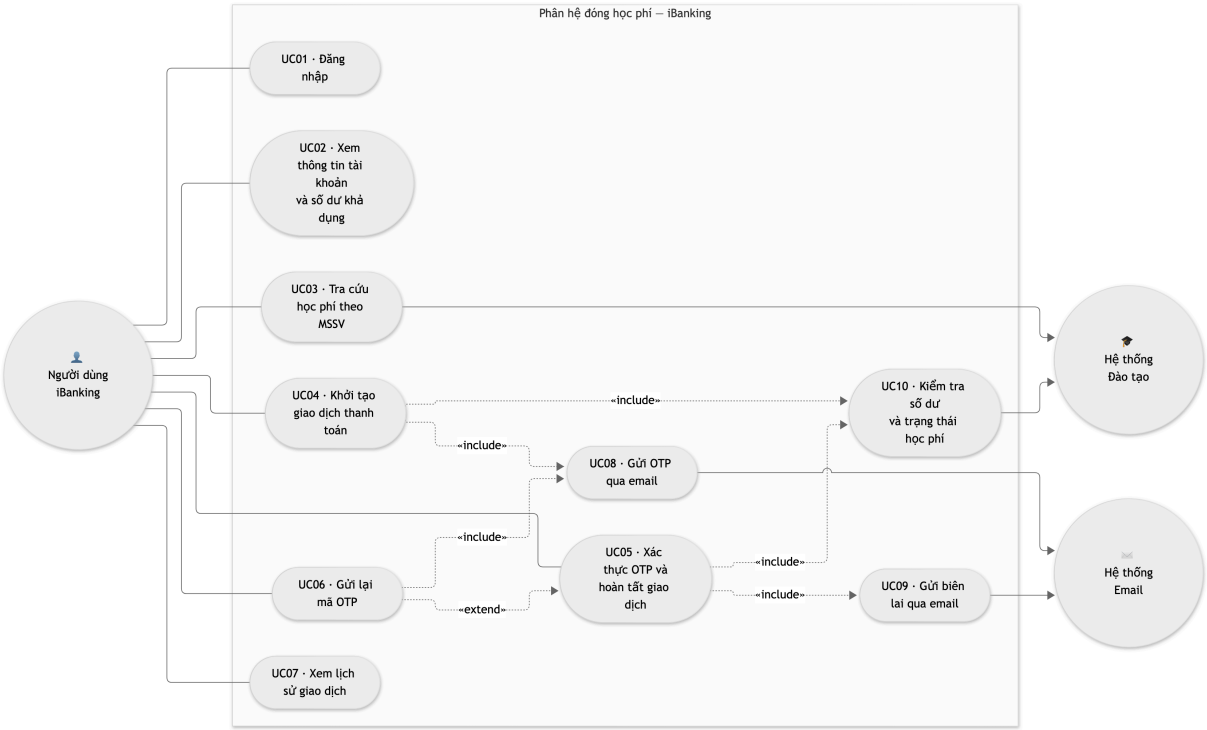
> Phân hệ đóng học phí — ứng dụng iBanking (TDTU) Các sơ đồ viết bằng Mermaid: xem trực tiếp trên GitHub/VS Code, hoặc dán vào https://mermaid.live để xuất PNG/SVG chèn vào báo cáo.

1.1. Tác nhân (Actor)

Tác nhân	Loại	Mô tả
Người dùng iBanking	Primary	Chủ tài khoản đã có sẵn trong hệ thống. Đăng nhập, tra cứu học phí và thanh toán cho chính mình **hoặc cho sinh viên khác**.
Hệ thống Email (SMTP)	Secondary / External	Nhận yêu cầu gửi mã OTP và biên lai giao dịch. Trong môi trường demo là MailHog.
Hệ thống Quản lý đào tạo	Secondary (mô phỏng)	Nguồn dữ liệu sinh viên và công nợ học phí. Ở đồ án được mô phỏng bằng `tuition-service` với dữ liệu mẫu + API admin.

Đề bài **không yêu cầu** chức năng đăng ký tài khoản, nên không có tác nhân quản trị người dùng.

1.2. Use Case Diagram



Hình 1. Sơ đồ phân tích và thiết kế

Đặc tả use case rút gọn

Mã	Use case	Tiền điều kiện	Hậu điều kiện thành công
UC01	Đăng nhập	Có tài khoản trong hệ thống	Nhận JWT, truy cập được chức năng thanh toán
UC02	Xem thông tin tài khoản	Đã đăng nhập	Hiển thị họ tên, SĐT, email, số dư khả dụng
UC03	Tra cứu học phí theo MSSV	Đã đăng nhập	Hiển thị thông tin sinh viên và khoản học phí còn nợ
UC04	Khởi tạo giao dịch	UC03 trả về khoản nợ, số dư ≥ số tiền, đã đồng ý điều khoản	Tạo giao dịch `PENDING_OTP`, gửi OTP qua email
UC05	Xác thực OTP, hoàn tất	Giao dịch `PENDING_OTP`, OTP còn hạn	Trừ tiền, gạch nợ học phí, ghi lịch sử, gửi biên lai
UC06	Gửi lại OTP	Giao dịch `PENDING_OTP`	Mã cũ bị thu hồi, mã mới được gửi
UC07	Xem lịch sử giao dịch	Đã đăng nhập	Danh sách giao dịch và biến động số dư

Đặc tả chi tiết — UC05 · Xác thực OTP và hoàn tất giao dịch

Luồng chính

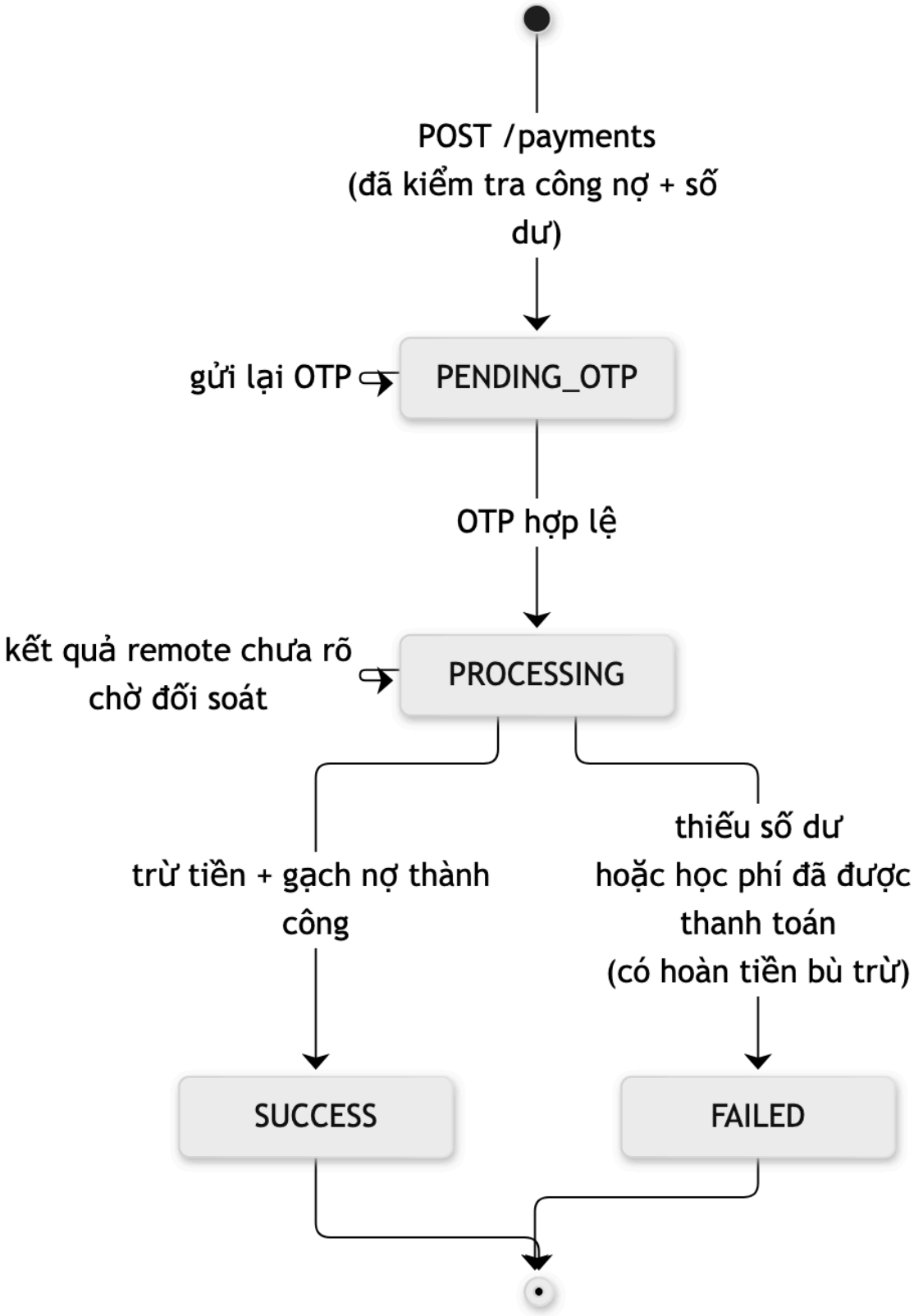
1. Người dùng nhập mã OTP 6 chữ số và bấm *Xác nhận*.
2. Hệ thống kiểm tra OTP: đúng giao dịch, còn hạn, chưa sử dụng.
3. Hệ thống khoá giao dịch, chuyển trạng thái `PENDING_OTP → PROCESSING`.
4. Hệ thống kiểm tra lại số dư khả dụng và **trừ tiền** người nộp.
5. Hệ thống **gạch nợ** khoản học phí tương ứng.
6. Hệ thống ghi lịch sử giao dịch, chuyển trạng thái `SUCCESS`.
7. Hệ thống gửi email biên lai và hiển thị kết quả.

Luồng thay thế / ngoại lệ

Mã	Tình huống	Xử lý
A1	OTP sai	Từ chối, tăng số lần thử, còn tối đa 5 lần
A2	OTP hết hạn (> 5 phút)	Từ chối, yêu cầu gửi lại mã (UC06)
A3	OTP đã dùng	Từ chối — mã chỉ dùng được một lần
A4	Nhập sai 5 lần	Khoá mã, buộc phát hành mã mới
A5	Số dư không đủ tại bước 4 (do giao dịch song song)	Giao dịch `FAILED`, **không** chạm vào học phí
A6	Học phí vừa được người khác thanh toán ở bước 5	**Hoàn tiền** đã trừ ở bước 4, giao dịch `FAILED`
A7	Gửi email biên lai lỗi	Giao dịch vẫn `SUCCESS`, ghi log lỗi gửi mail
A8	Timeout/mất phản hồi khi trừ tiền, gạch nợ hoặc hoàn tiền	Giữ `PROCESSING`, chờ job đối soát đọc ledger/công nợ; không suy đoán thất bại từ lỗi mạng

---

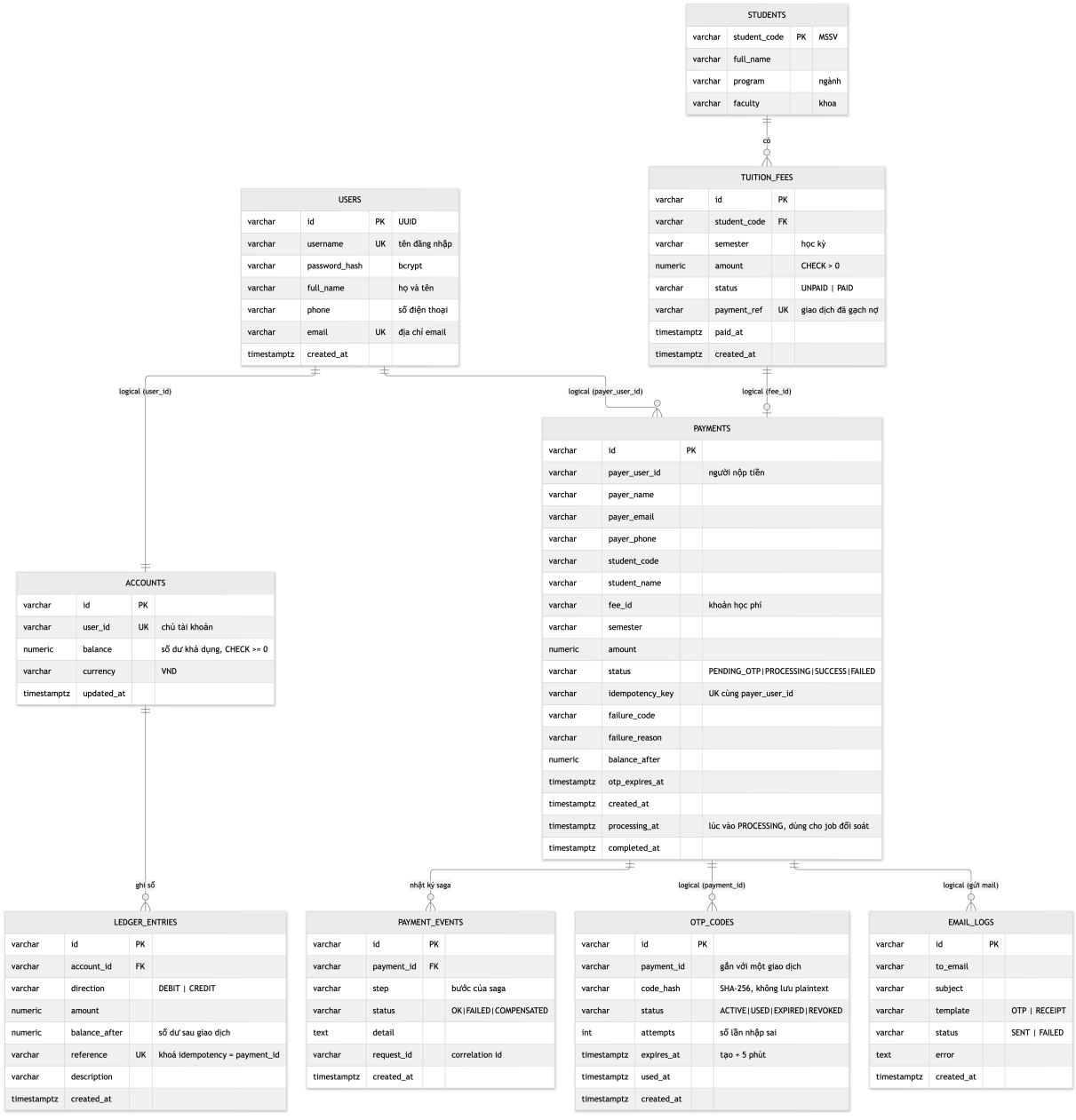
1.3. Sơ đồ trạng thái của giao dịch



Hình 2. Sơ đồ phân tích và thiết kế

1.4. Entity Relationship Diagram

Hệ thống theo kiến trúc microservices nên dữ liệu được tách theo **database-per-service**. ERD dưới đây là **mô hình logic toàn phân hệ**; khung màu cho biết entity thuộc service nào. Quan hệ **nét liền** là khoá ngoại thật trong cùng một database; quan hệ **in nghiêng ghi chú "logical"** là tham chiếu logic xuyên service (không có FK vật lý, đúng nguyên tắc service tự trị dữ liệu).



Hình 3. Sơ đồ phân tích và thiết kế

Phân bổ entity theo service

Database	Service sở hữu	Entity
`authdb`	auth-service	`USERS`

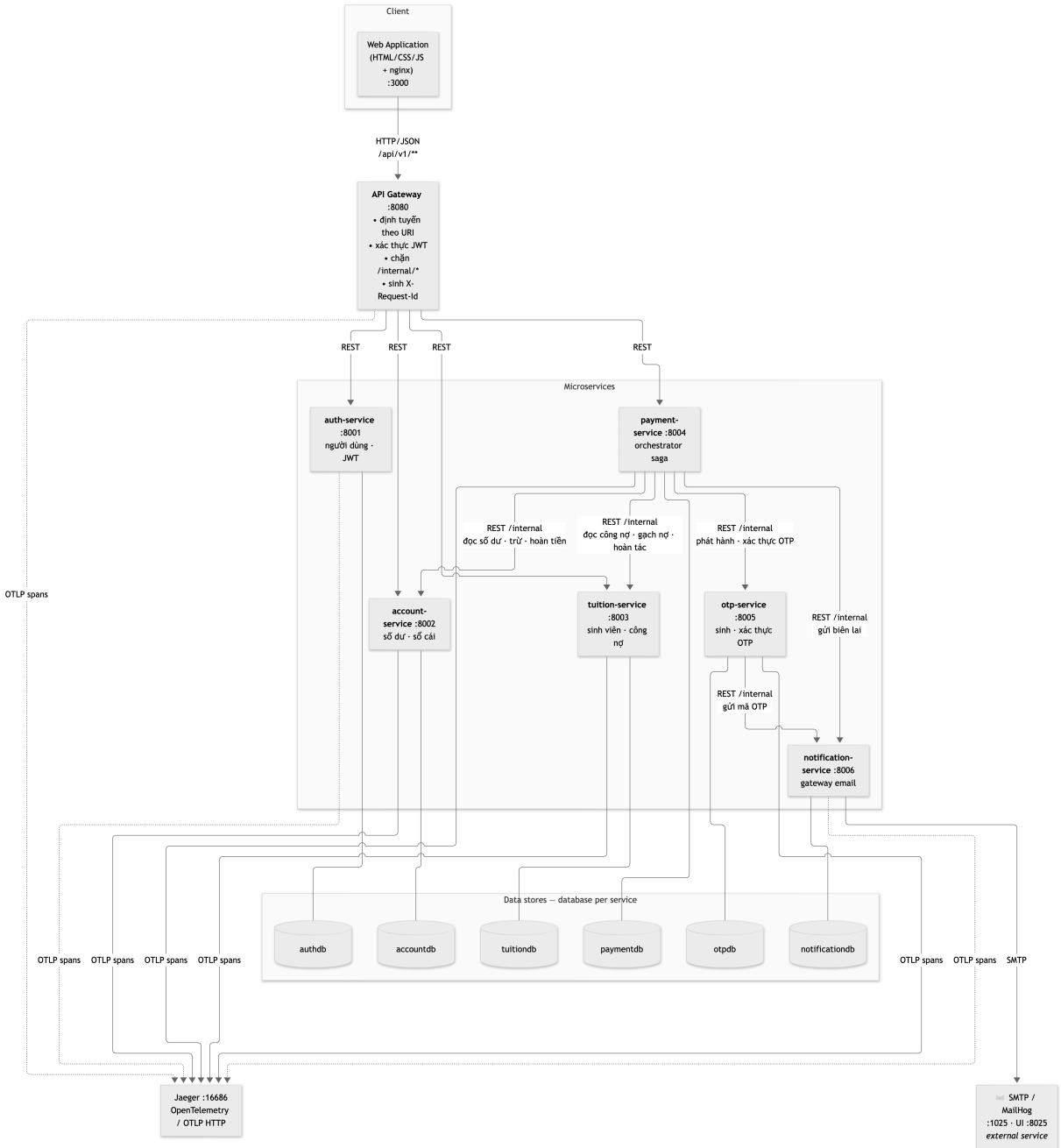
Database	Service sở hữu	Entity
accountdb	account-service	ACCOUNTS, LEDGER_ENTRIES
tuitiondb	tuition-service	STUDENTS, TUITION_FEES
paymentdb	payment-service	PAYMENTS, PAYMENT_EVENTS
otpdb	otp-service	OTP_CODES
notificationdb	notification-service	EMAIL_LOGS

Ràng buộc dữ liệu quan trọng

Ràng buộc	Kiểu	Mục đích nghiệp vụ
accounts.balance >= 0	CHECK	Chốt chặn cuối: không bao giờ chi vượt số dư
ledger_entries.reference	UNIQUE	Idempotency — một giao dịch chỉ ghi sổ một lần
tuition_fees (student_code, semester)	UNIQUE	Mỗi học kỳ một khoản học phí
tuition_fees.payment_ref	UNIQUE	Một giao dịch chỉ gạch nợ được một khoản
tuition_fees.status IN ('UNPAID','PAID')	CHECK	Không cho thanh toán một phần
payments (payer_user_id, idempotency_key)	UNIQUE	Chống tạo trùng giao dịch khi bấm nhiều lần
payments.fee_id WHERE status='SUCCESS'	UNIQUE (partial)	**Một khoản học phí chỉ có một giao dịch thành công**
otp_codes.payment_id WHERE status='ACTIVE'	UNIQUE (partial)	Mỗi giao dịch chỉ một OTP còn hiệu lực
otp_codes.code_hash WHERE status='ACTIVE'	UNIQUE (partial)	**OTP không trùng với OTP đang hiệu lực của giao dịch khác**

2. Thiết kế kiến trúc Microservices

2.1. Microservices Architecture Diagram



Hình 4. Sơ đồ phân tích và thiết kế

2.2. Phân rã service và trách nhiệm

Service	Trách nhiệm (bounded context)	Sở hữu dữ liệu	KHÔNG làm
api-gateway	Điểm vào duy nhất; định tuyến theo tiền tố URI; xác thực JWT sớm; chặn endpoint `/internal/*`; sinh & truyền `X-Request-Id`; CORS	không có DB	Không chứa bất kỳ logic nghiệp vụ nào
auth-service	Xác thực username/password (bcrypt), phát hành JWT, cung cấp hồ sơ người dùng	`users`	Không giữ số dư
account-service	Số dư khả dụng, trừ/hoàn tiền **có khoá dòng**, sổ cái lịch sử giao dịch	`accounts`, `ledger_entries`	Không biết học phí là gì
tuition-service	Danh mục sinh viên, công nợ học phí, tra cứu theo MSSV, gạch nợ/huỷ gạch nợ	`students`, `tuition_fees`	Không biết tiền ở đâu ra
payment-service	**Orchestrator**: kiểm tra điều kiện, tạo giao dịch, điều phối OTP → trừ tiền → gạch nợ → biên lai, bù trừ khi lỗi, lưu saga trace	`payments`, `payment_events`	Không tự sửa số dư hay học phí
otp-service	Sinh OTP không trùng, gắn với giao dịch, hạn 5 phút, dùng một lần, đếm số lần sai	`otp_codes`	Không biết nghiệp vụ thanh toán
notification-service	Cổng ra duy nhất tới email; render template OTP/biên lai; nhật ký gửi mail	`email_logs`	Không sinh OTP

Nguyên tắc phân rã: mỗi service là một *bounded context* có dữ liệu riêng và một lý do duy nhất để thay đổi. Ví dụ đổi chính sách OTP (độ dài mã, thời hạn) chỉ động vào `otp-service`; đổi nhà cung cấp email chỉ động vào `notification-service`.

2.3. Giao tiếp giữa các service

Toàn bộ giao tiếp là **REST/JSON đồng bộ trên HTTP** trong mạng nội bộ Docker.

Bên gọi	Bên nhận	Endpoint	Mục đích	Đồng bộ?
web	api-gateway	`/api/v1/**`	Mọi thao tác người dùng	✔

Bên gọi	Bên nhận	Endpoint	Mục đích	Đồng bộ?
api-gateway	auth / account / tuition / payment	`/api/v1/**`	Proxy sau khi xác thực JWT	✔
payment-service	tuition-service	`GET /internal/tuitions/students/{mssv}`	Đọc công nợ để dựng giao dịch	✔
payment-service	account-service	`GET /internal/accounts/{user_id}`	Kiểm tra số dư trước khi gửi OTP	✔
payment-service	otp-service	`POST /internal/otp/issue` · `/verify`	Phát hành & xác thực OTP	✔
payment-service	account-service	`POST .../debit` · `.../credit`	Trừ tiền / **hoàn tiền bù trừ**	✔
payment-service	tuition-service	`POST .../fees/{id}/pay` · `/revert`	Gạch nợ / **huỷ gạch nợ bù trừ**	✔
payment-service	notification-service	`POST /internal/notifications/email`	Gửi biên lai (best-effort)	✔
otp-service	notification-service	`POST /internal/notifications/email`	Gửi mã OTP	✔
notification-service	SMTP (MailHog)	giao thức SMTP	Gửi email thật	✔

Bảo mật lời gọi nội bộ. Endpoint `/internal/*` yêu cầu header `X-Internal-Token`; gateway chặn đường dẫn này trên cổng 8080. Compose vẫn mở cổng 8001–8006 trực tiếp ra host để dev/demo, nên chặn tại gateway không thay thế cô lập mạng. Không triển khai cấu hình demo và token mặc định lên mạng công cộng.

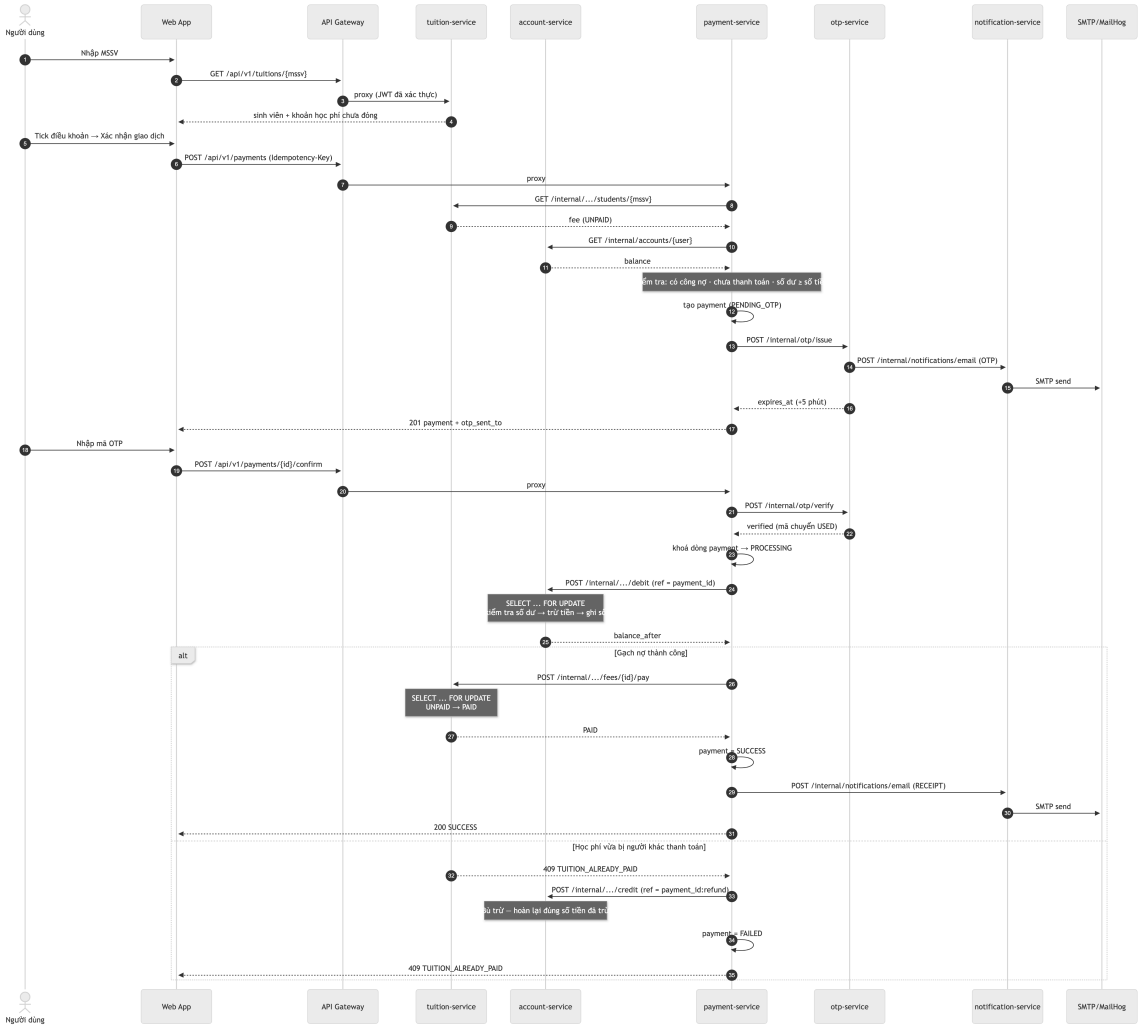
Truy vết phân tán bằng OpenTelemetry. Điểm `instrument(app, service)` dùng chung gắn instrumentation FastAPI và HTTPX cho gateway cùng sáu service. W3C `traceparent` truyền qua các lời gọi REST; server/client span ghi quan hệ cha–con, thời gian và trạng thái HTTP. `BatchSpanProcessor` xuất OTLP HTTP tới Jaeger, giao diện ở http://localhost:16686. Không thu body chứa OTP, mật khẩu hay JWT vào span.

`X-Request-Id` vẫn được truyền và lưu vào `payment_events.request_id`. JSON log bổ sung `trace_id` và `span_id` để chuyển từ log sang waterfall Jaeger. Các request tạo payment và xác nhận OTP là các trace riêng; `payment_id` liên kết chúng ở mức nghiệp vụ. Job nền tạo span `payment.reconcile` riêng với payment ID/action, không giả định vẫn có context của request trước crash.

```
docker compose logs --no-log-prefix | grep '<request-id-hoặc-trace-id>'
```

Jaeger dùng bộ nhớ cho demo: restart làm mất trace. Tracing export bất đồng bộ, collector không phải dependency nghiệp vụ; log và sổ cái vẫn hoạt động khi Jaeger không sẵn sàng. Chưa instrument SQL/SMTP hay lưu trữ trace dài hạn.

2.4. Sequence Diagram — luồng thanh toán đầy đủ



Hình 5. Sơ đồ phân tích và thiết kế

2.5. Giải thích các quyết định kiến trúc

Vì sao phân rã như trên?

Ranh giới service bám theo **năng lực nghiệp vụ (business capability)**, không theo tầng kỹ thuật: danh tính · tiền · công nợ · điều phối giao dịch · xác thực OTP · thông báo. Đây cũng chính là ranh giới thay đổi trong thực tế ngân hàng (đổi core thẻ, đổi nhà cung cấp

SMS/email, đổi hệ đào tạo) nên mỗi thay đổi chỉ ảnh hưởng một service.

Vì sao database-per-service?

Không service nào biết connection string của service khác; muốn dữ liệu của nhau thì phải đi qua API. Nhờ vậy: - lược đồ dữ liệu của một service thay đổi mà không làm vỡ service khác; - không có join xuyên nghiệp vụ — tránh "monolith phân tán".

Demo dùng **một Postgres instance với 6 database riêng** thay vì 6 instance, để tiết kiệm tài nguyên máy chấm. Ranh giới logic vẫn nguyên vẹn và có thể tách thành 6 instance mà không sửa code (chỉ đổi `DATABASE_URL`).

Vì sao REST đồng bộ, không dùng message broker?

Luồng thanh toán là **request–response có tính tương tác**: người dùng đứng chờ kết quả để biết giao dịch thành công hay không. Đồng bộ cho phép trả lỗi ngay và giữ code đơn giản. Chỗ *nên* bất đồng bộ là gửi email — đã được cô lập trong `notification-service`, và lỗi gửi biên lai **không** làm hỏng giao dịch đã thành công. Khi cần chịu tải cao hơn, chỉ việc thay lời gọi HTTP tới notification bằng publish message, không đụng đến nghiệp vụ.

Vì sao chọn orchestration saga thay vì choreography?

Giao dịch tài chính cần một nơi duy nhất trả lời được câu hỏi *"giao dịch này đang ở đâu, vì sao hỏng"*. `payment-service` giữ state machine và ghi `payment_events` cho từng bước; việc bù trừ (hoàn tiền) là tường minh, dễ demo và dễ audit. Choreography bằng event sẽ phân tán logic bù trừ ra nhiều service và rất khó giải thích khi bảo vệ đồ án.

Vì sao có API Gateway?

- Client chỉ cần biết **một** địa chỉ, không phải 6 cổng.
- Xác thực JWT tập trung, fail-fast trước khi chạm tới nghiệp vụ.
- Chặn `/internal/*` trên đường vào của web; từng service vẫn kiểm tra token nội bộ.
- Đã tích hợp tracing OpenTelemetry và log JSON; có thể bổ sung rate-limit tập trung.

Các service **vẫn tự verify JWT** (zero-trust) chứ không tin header do gateway gắn vào, phòng trường hợp ai đó gọi thẳng service trong mạng nội bộ.

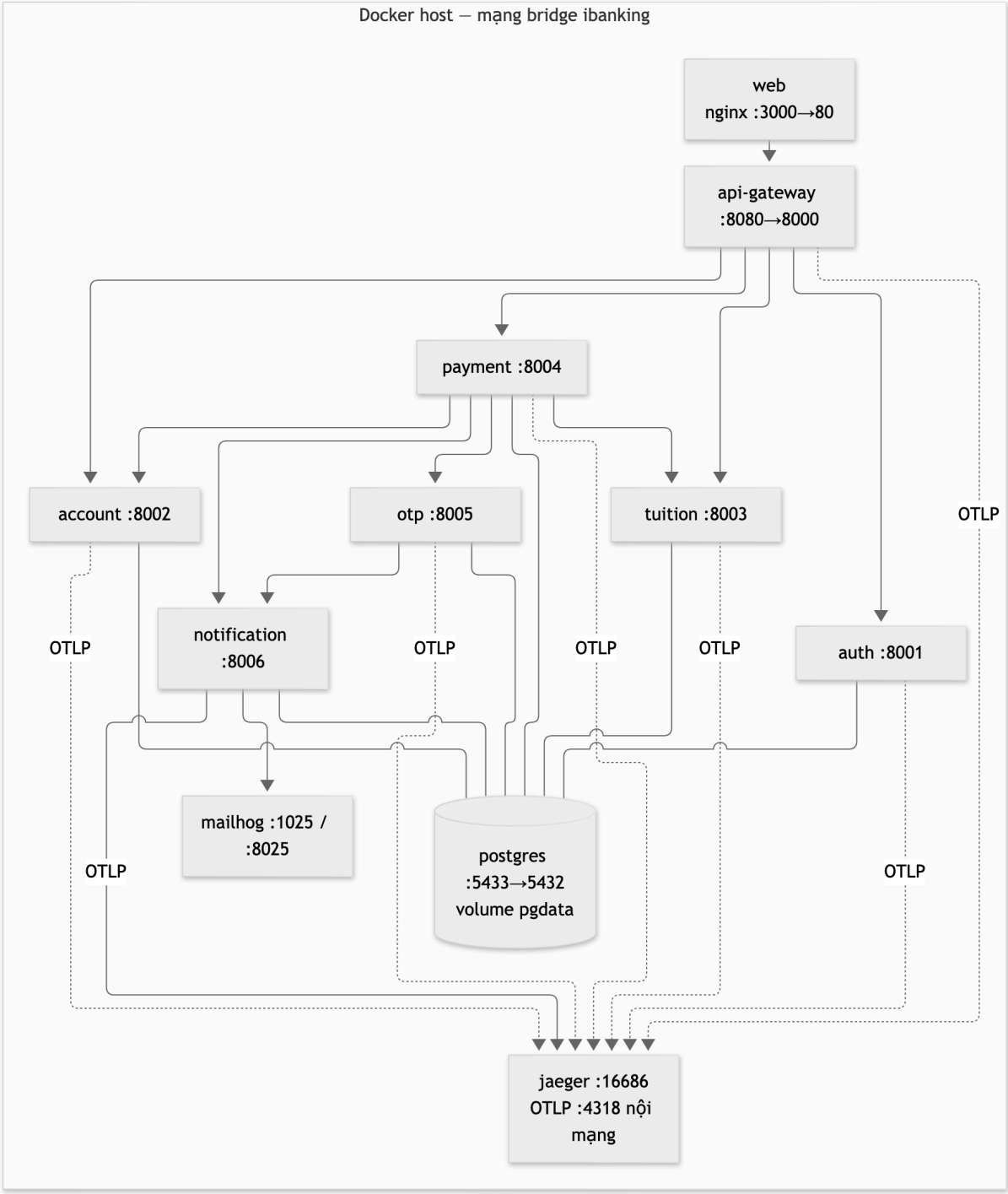
Đánh đổi đã chấp nhận (và hướng nâng cấp)

Quyết định	Đánh đổi	Khi nào cần nâng cấp
REST đồng bộ cho saga	payment-service phụ thuộc tính sẵn sàng của 4 service	Tải cao → chuyển bước gửi mail sang message broker + outbox pattern

Quyết định	Đánh đổi	Khi nào cần nâng cấp
Một Postgres instance, 6 database	Vẫn là một điểm hỏng chung ở tầng hạ tầng	Production → tách instance/cluster riêng cho từng service
Saga + job đối soát	Nhất quán cuối cùng; phục hồi phụ thuộc service trở lại; giữ connection khi khóa payment qua HTTP	Tải cao hoặc remote RPC trì hoãn dài → lease/fencing, outbox và vận hành đối soát
JWT dùng shared secret HS256	Mọi service phải biết secret	Nhiều team → chuyển RS256, service chỉ giữ public key
Thư viện `common/` dùng chung	Thay đổi common ảnh hưởng nhiều service	Tách thành package versioned, mỗi service nâng cấp độc lập

---

2.6. Deployment view



Hình 6. Sơ đồ phân tích và thiết kế

Mỗi service là **một container độc lập**, cùng một `Dockerfile` tham số hoá bằng `--build-arg SERVICE`. `docker-compose.yml` khai báo `healthcheck` cho từng service và `depends_on: condition: service_healthy`, nên thứ tự khởi động luôn đúng: postgres → notification → otp → (account, tuition) → payment → gateway → web.

3. Thiết kế REST API

Tài liệu tương tác (OpenAPI/Swagger) được sinh tự động cho từng service:

Service	Swagger UI	OpenAPI JSON
API Gateway	http://localhost:8080/docs	`/openapi.json`
auth-service	http://localhost:8001/docs	`/openapi.json`
account-service	http://localhost:8002/docs	`/openapi.json`
tuition-service	http://localhost:8003/docs	`/openapi.json`
payment-service	http://localhost:8004/docs	`/openapi.json`
otp-service	http://localhost:8005/docs	`/openapi.json`
notification-service	http://localhost:8006/docs	`/openapi.json`

`GET http://localhost:8080/services` trả về danh sách toàn bộ service kèm link tài liệu.

3.1. Quy ước chung

Hạng mục	Quy ước
Base URL	`http://localhost:8080/api/v1` (qua API Gateway)
Versioning	Trên URI: `/api/v1/...`
Định dạng	JSON, `Content-Type: application/json`, UTF-8
Xác thực	`Authorization: Bearer <JWT>` cho mọi API trừ `POST /auth/login`
Idempotency	Header `Idempotency-Key` (UUID) ở `POST /payments`
Truy vết	`X-Request-Id` trong response; OpenTelemetry truyền W3C `traceparent` giữa các service, JSON log có `trace_id/span_id`
Tài nguyên	Danh từ số nhiều: `/accounts`, `/tuitions`, `/payments`
Phân trang	`?limit=n` (mặc định 20–50, trần 100–200)
Nội bộ	`/internal/*` — chỉ service gọi nhau, yêu cầu `X-Internal-Token`, gateway chặn

Mẫu lỗi thống nhất

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Số dư khả dụng không đủ để thực hiện giao dịch.",
    "details": { "balance": "500000.00", "required": "9500000.00" },
    "request_id": "3f9a1c2b7d4e5a60"
  }
}
```

Quy ước HTTP Status Code

Code	Dùng khi
`200 OK`	Đọc/ghi thành công
`201 Created`	Tạo giao dịch, tạo khoản học phí
`400 Bad Request`	OTP sai/hết hạn/bị khoá, chưa đồng ý điều khoản
`401 Unauthorized`	Thiếu token, token sai hoặc hết hạn, sai mật khẩu
`403 Forbidden`	Truy cập giao dịch của người khác, gọi `/internal/*` từ ngoài
`404 Not Found`	MSSV/giao dịch/tài khoản không tồn tại
`409 Conflict`	Xung đột trạng thái: hết số dư, học phí đã thanh toán, không còn công nợ
`422 Unprocessable Entity`	Sai kiểu/định dạng dữ liệu (validation của schema)
`502 Bad Gateway`	Gửi email thất bại
`503 Service Unavailable`	Service phụ thuộc không phản hồi

3.2. API công khai (qua API Gateway)

3.2.1. Authentication — `auth-service`

#	Method	URI	Mô tả	Auth
A1	`POST`	`/api/v1/auth/login`	Đăng nhập	❌
A2	`GET`	`/api/v1/auth/me`	Thông tin người dùng đang đăng nhập	✔

A1 · POST /api/v1/auth/login

Request

```
{ "username": "minhnguyen", "password": "123456" }
```

Response `200 OK`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "id": "11111111-1111-1111-1111-111111111111",
    "username": "minhnguyen",
    "full_name": "Nguyễn Văn Minh",
    "phone": "0901234567",
    "email": "minhnguyen@tdtu.edu.vn"
  }
}
```

Status	Code	Tình huống
401	`INVALID_CREDENTIALS`	Sai username hoặc password
422	`VALIDATION_ERROR`	Thiếu trường, password < 6 ký tự

3.2.2. Account — `account-service`

#	Method	URI	Mô tả	Auth
B1	`GET`	`/api/v1/accounts/me`	Số dư khả dụng	✔
B2	`GET`	`/api/v1/accounts/me/transactions?limit=50`	Lịch sử biến động số dư	✔

B1 → `200 OK`

```
{ "account_id": "a0000001-...", "user_id": "11111111-...", "balance": "50000000.00",
"currency": "VND" }
```

B2 → `200 OK`

```
{
  "items": [{
    "id": "9c1f...", "direction": "DEBIT", "amount": "9500000.00",
    "balance_after": "40500000.00", "reference": "4b2e-...",
    "description": "Thanh toán học phí HK1/2026-2027 - MSSV 522H0001",
    "created_at": "2026-09-14T08:12:31.442Z"
  }],
  "total": 1
}
```

Status	Code	Tình huống
401	`UNAUTHORIZED` / `TOKEN_EXPIRED`	Không có/hết hạn token
404	`ACCOUNT_NOT_FOUND`	Người dùng chưa có tài khoản thanh toán

3.2.3. Tuition — `tuition-service`

#	Method	URI	Mô tả	Auth
C1	`GET`	`/api/v1/tuitions/{student_code}`	Tra cứu sinh viên + công nợ học phí	✔

C1 → `200 OK`

```
{
  "student": {
    "student_code": "522H0001", "full_name": "Nguyễn Văn Minh",
    "program": "Kỹ thuật phần mềm", "faculty": "Công nghệ thông tin"
  },
  "outstanding_fee": {
    "id": "f0000001-...", "student_code": "522H0001", "semester": "HK1/2026-2027",
    "amount": "9500000.00", "status": "UNPAID", "payment_ref": null, "paid_at": null
  },
  "fees": [ "... toàn bộ khoản học phí của sinh viên ..." ]
}
```

`outstanding_fee = null` nghĩa là sinh viên không còn khoản phải đóng.

Status	Code	Tình huống
404	`STUDENT_NOT_FOUND`	MSSV không tồn tại

3.2.4. Payment — `payment-service`

#	Method	URI	Mô tả	Auth
D1	`POST`	`/api/v1/payments`	Khởi tạo giao dịch + gửi OTP	✔
D2	`POST`	`/api/v1/payments/{id}/confirm`	Xác thực OTP và hoàn tất giao dịch	✔
D3	`POST`	`/api/v1/payments/{id}/otp/resend`	Gửi lại mã OTP	✔
D4	`GET`	`/api/v1/payments/{id}`	Chi tiết giao dịch	✔
D5	`GET`	`/api/v1/payments/{id}/events`	Nhật ký các bước xử lý (saga trace)	✔
D6	`GET`	`/api/v1/payments?limit=20`	Lịch sử giao dịch của người dùng	✔

D1 · POST /api/v1/payments

Headers: `Authorization: Bearer <JWT>`, `Idempotency-Key: <uuid>` *(khuyến nghị)*

Request

```
{ "student_code": "522H0001", "accept_terms": true }
```

Response `201 Created`

```
{
  "payment": {
    "id": "4b2e8f11-...", "status": "PENDING_OTP",
    "payer": { "name": "Nguyễn Văn Minh", "email": "minhnguyen@tdtu.edu.vn", "phone":
"0901234567" },
    "student_code": "522H0001", "student_name": "Nguyễn Văn Minh",
    "fee_id": "f0000001-...", "semester": "HK1/2026-2027", "amount": "9500000.00",
    "otp_expires_at": "2026-09-14T08:17:31.442Z", "created_at": "2026-09-
14T08:12:31.442Z"
  },
```

```
  "otp_sent_to": "mi*******@tdtu.edu.vn",
  "otp_ttl_seconds": 300
}
```

Status	Code	Tình huống
400	`TERMS_NOT_ACCEPTED`	Chưa đồng ý điều khoản
404	`STUDENT_NOT_FOUND`	MSSV không tồn tại
409	`NO_OUTSTANDING_TUITION`	Học phí đã đóng / không có công nợ
409	`INSUFFICIENT_FUNDS`	Số dư < số tiền cần thanh toán
503	`SERVICE_UNAVAILABLE`	Không gọi được tuition/account/otp service

D2 · POST /api/v1/payments/{id}/confirm

Request

```
{ "otp_code": "482913" }
```

Response `200 OK` — giao dịch hoàn tất

```
{
  "id": "4b2e8f11-...", "status": "SUCCESS",
  "payer": { "name": "Nguyễn Văn Minh", "email": "minhnguyen@tdtu.edu.vn", "phone":
"0901234567" },
  "student_code": "522H0001", "student_name": "Nguyễn Văn Minh",
  "semester": "HK1/2026-2027", "amount": "9500000.00", "balance_after":
"40500000.00",
  "completed_at": "2026-09-14T08:13:02.887Z"
}
```

Status	Code	Tình huống
400	`OTP_INVALID`	Sai mã (kèm `details.remaining_attempts`)
400	`OTP_EXPIRED`	Quá 5 phút
400	`OTP_NOT_FOUND`	Không có mã hiệu lực (đã dùng / đã thu hồi)
400	`OTP_LOCKED`	Nhập sai quá 5 lần

Status	Code	Tình huống
403	`FORBIDDEN`	Giao dịch không thuộc người dùng hiện tại
404	`PAYMENT_NOT_FOUND`	Giao dịch không tồn tại
409	`INSUFFICIENT_FUNDS`	Số dư không đủ tại thời điểm trừ tiền (giao dịch song song)
409	`TUITION_ALREADY_PAID`	Học phí vừa được người khác thanh toán — **đã tự động hoàn tiền**
409	`PAYMENT_IN_PROGRESS`	Giao dịch đang xử lý
422	`VALIDATION_ERROR`	`otp_code` không phải 6 chữ số
503	`SERVICE_UNAVAILABLE`	Mất kết nối/phản hồi; giao dịch có thể vẫn `PROCESSING` để đối soát

Lỗi kỹ thuật hoặc mã lỗi upstream chưa xác định không được hiểu là "chưa trừ tiền". Client cần đọc `GET /payments/{id}` hoặc lịch sử để lấy kết quả cuối, không tạo payment mới chỉ vì timeout. Nếu thao tác hoàn tiền cũng lỗi, payment giữ `PROCESSING` thay vì giả báo đã hoàn.

Gọi lại `confirm` cho giao dịch đã `SUCCESS` trả về `200` với chính giao dịch đó (idempotent), không trừ tiền lần hai.

D5 → `200 OK`

```
[
  { "step": "PAYMENT_CREATED",    "status": "OK", "detail": "fee=f000...
amount=9500000.00" },
  { "step": "OTP_ISSUED",         "status": "OK", "detail": "hết hạn 2026-09-
14T08:17:31Z" },
  { "step": "OTP_VERIFIED",       "status": "OK", "detail": null },
  { "step": "DEBIT",              "status": "OK", "detail": "số dư còn 40500000.00"
},
  { "step": "MARK_TUITION_PAID",  "status": "OK", "detail": null },
  { "step": "PAYMENT_COMPLETED",  "status": "OK", "detail": null },
  { "step": "RECEIPT_EMAIL",      "status": "OK", "detail": null }
]
```

3.2.5. Hệ thống

Method	URI	Mô tả
GET	/health	Liveness của gateway
GET	/health/services	Health tổng hợp; 503 nếu có service DOWN
GET	/services	Danh sách service + link Swagger

3.3. API nội bộ (service-to-service)

Yêu cầu header X-Internal-Token. **Không** truy cập được qua gateway.

Service	Method	URI	Mục đích
account	GET	/internal/accounts/{user_id}	Đọc số dư
account	GET	/internal/accounts/{user_id}/entries/{reference}	Đọc bút toán đúng tài khoản; 404 nếu không có hoặc thuộc tài khoản khác
payment	POST	/internal/payments/reconcile?stuck_after_seconds=60	Đối soát một lượt; ngưỡng ≥ 0, mặc định lấy cấu hình; trả scanned và resolved
account	POST	/internal/accounts/{user_id}/debit	Trừ tiền (idempotent theo reference)
account	POST	/internal/accounts/{user_id}/credit	Hoàn tiền — bù trừ saga
tuition	GET	/internal/tuitions/students/{student_code}	Đọc công nợ theo MSSV
tuition	GET	/internal/tuitions/fees/{fee_id}	Đọc một khoản học phí
tuition	POST	/internal/tuitions/fees	Tạo sinh viên/khoản học phí (admin, phục vụ test)
tuition	POST	/internal/tuitions/fees/{fee_id}/pay	Gạch nợ — chỉ thành công một lần
tuition	POST	/internal/tuitions/fees/{fee_id}/revert	Huỷ gạch nợ — bù trừ saga
otp	POST	/internal/otp/issue	Phát hành OTP + gửi email

Service	Method	URI	Mục đích
otp	`POST`	`/internal/otp/verify`	Xác thực OTP (một lần duy nhất)
otp	`GET`	`/internal/otp/{payment_id}/status`	Trạng thái/hạn dùng của OTP
notification	`POST`	`/internal/notifications/email`	Gửi email theo template `OTP/RECEIPT`
notification	`GET`	`/internal/notifications`	Nhật ký email đã gửi
auth	`GET`	`/internal/users/{user_id}`	Hồ sơ người dùng

Ví dụ `POST /internal/accounts/{user_id}/debit`

```
{ "amount": "9500000.00", "reference": "4b2e8f11-...", "description": "Thanh toán học phí HK1/2026-2027 - MSSV 522H0001" }
```

```
{ "ledger_id": "9c1f...", "balance_after": "40500000.00", "idempotent_replay": false }
```

`409 INSUFFICIENT_FUNDS` nếu số dư không đủ. Gọi lại cùng `reference` trả `idempotent_replay: true` và **không** trừ tiền thêm. Replay chỉ được công nhận khi trùng cả tài khoản, chiều và số tiền; `reference` đã thuộc về bút toán khác trả `409 REFERENCE_CONFLICT` (áp dụng cho cả `debit` lẫn `credit`).

3.4. Validation và error handling

Tầng	Kiểm tra	Ví dụ
Schema (Pydantic)	Kiểu, độ dài, regex, số dương	`otp_code` khớp `^\d{6}$`; `amount > 0`, 2 chữ số thập phân
Nghiệp vụ (service)	Trạng thái hợp lệ	Học phí `UNPAID`, giao dịch `PENDING_OTP`, OTP còn hạn
Quyền	Chủ sở hữu tài nguyên	Chỉ người tạo giao dịch mới xem/xác nhận được
Cơ sở dữ liệu	CHECK / UNIQUE / partial index	`balance >= 0`, một `SUCCESS` cho mỗi `fee_id`

Mọi ngoại lệ đều đi qua `install_error_handlers()` (`services/common/errors.py`) nên client **luôn** nhận đúng một cấu trúc lỗi, kể cả lỗi validation của framework hay lỗi 500 ngoài dự kiến. Lỗi nghiệp vụ từ service nội bộ được `call_service()` giữ nguyên mã lỗi khi truyền ngược lên client — ví dụ `INSUFFICIENT_FUNDS` của account-service tới thẳng trình duyệt thay vì biến thành `500`.

4. Xử lý Transaction và Concurrency

> Mục tiêu: sau khi giao dịch kết thúc, **số dư · trạng thái học phí · lịch sử giao dịch** phải phản ánh đúng một kết quả cuối cùng duy nhất, kể cả khi nhiều giao dịch chạy chồng lên nhau.

4.1. Hai tầng nhất quán

Phạm vi	Cơ chế	Đảm bảo
Trong một service	Transaction ACID của PostgreSQL + `SELECT ... FOR UPDATE` + CHECK/UNIQUE constraint	Trừ tiền và ghi sổ là **nguyên tử**; gạch nợ học phí là **nguyên tử**
Xuyên nhiều service	**Saga có bù trừ** và job đối soát do `payment-service` điều phối, các bước ghi tiền idempotent	Hội tụ về kết quả nhất quán khi các dịch vụ phụ thuộc hoạt động trở lại; có trạng thái trung gian

Không dùng two-phase commit (XA). Saga chấp nhận *eventual consistency*: tiền có thể đã trừ khi học phí chưa gạch nợ. Luồng bình thường thường nhanh, nhưng khi lỗi mạng hoặc tiến trình sập, thời gian phục hồi phụ thuộc ngưỡng đối soát và tính sẵn sàng của các service; không cam kết dưới một giây.

4.2. Transaction cục bộ

Trừ tiền — `account-service`

```
# services/account-service/app/main.py
account = db.execute(
    select(Account).where(Account.user_id == user_id).with_for_update()    # ① khoá dòng
).scalar_one_or_none()

if existing := replay_of(db, account, req, direction):                     # ② idempotency
    return existing_result

if account.balance < amount:                                               # ③ kiểm tra trong khoá
    raise APIError(409, "INSUFFICIENT_FUNDS", ...)
```

```
account.balance -= amount                                                    # ④ trừ
tiền
db.add(LedgerEntry(..., balance_after=account.balance, reference=req.reference))
db.commit()                                                                  # ⑤ nhả
khoá
```

- ① `SELECT ... FOR UPDATE` khoá **dòng tài khoản** tới khi commit. Giao dịch thứ hai trên cùng tài khoản bị chặn ở bước ① cho đến khi giao dịch đầu kết thúc, nên bước ③ luôn đọc số dư mới nhất — **không có read-modify-write race**.
- ② Kiểm tra `reference` ngay trong khoá: gọi lại cùng một giao dịch không trừ tiền lần hai. `reference` là UNIQUE **toàn hệ thống**, nên chỉ coi là replay khi bút toán cũ trùng cả **tài khoản, chiều ghi nợ/ghi có và số tiền**; lệch bất kỳ yếu tố nào thì trả `409 REFERENCE_CONFLICT` thay vì báo thành công giả. Tra theo mỗi `reference` sẽ bắt trúng bút toán của tài khoản khác: job đối soát nhận `idempotent_replay` rồi chốt "đã hoàn tiền" cho một giao dịch thực tế chưa hề được hoàn.
- ⑤ Số dư và bút toán sổ cái được commit **cùng một transaction** → không bao giờ lệch nhau.

Ba lớp bảo vệ chồng lên nhau: **khoá dòng** (ngăn race) → **UNIQUE** `reference` (ngăn ghi trùng khi hai request cùng reference chạy song song) → **CHECK** `balance >= 0` (chốt chặn cuối ở tầng database, kể cả khi code sai).

Gạch nợ học phí — `tuition-service`

```
fee = db.execute(
    select(TuitionFee).where(TuitionFee.id == fee_id).with_for_update()
).scalar_one_or_none()

if fee.status == PAID:
    if fee.payment_ref == req.payment_ref:
        return idempotent_ok                      # chính giao dịch này retry
    raise APIError(409, "TUITION_ALREADY_PAID")   # giao dịch khác đã thắng

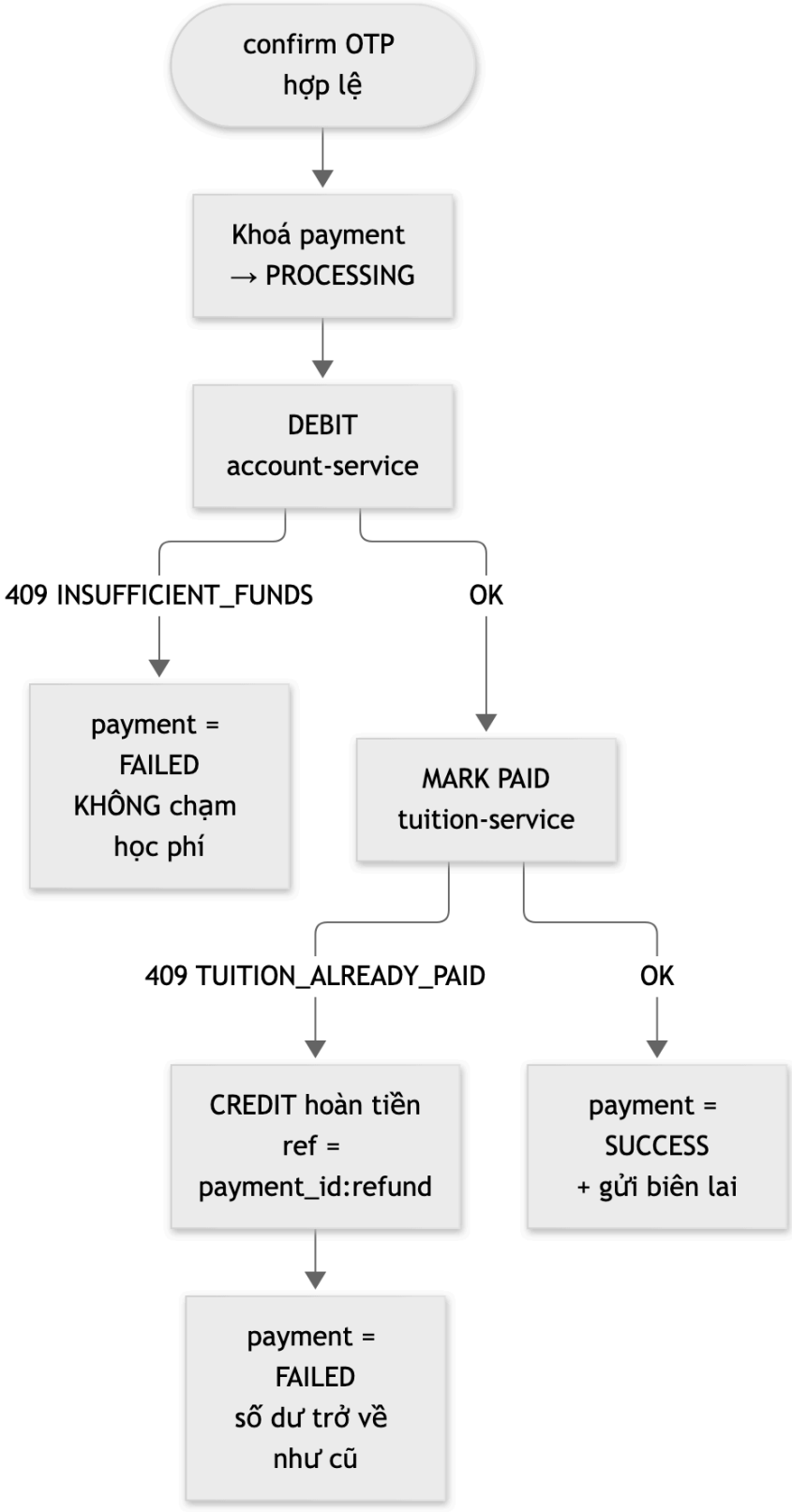
fee.status, fee.payment_ref, fee.paid_at = PAID, req.payment_ref, now()
db.commit()
```

Chuyển trạng thái `UNPAID → PAID` xảy ra **bên trong khoá dòng**, nên dù hai request vào cùng lúc, chỉ một request đọc được `UNPAID`.

Mức cô lập (isolation level)

Dùng **READ COMMITTED** (mặc định của PostgreSQL) thay vì SERIALIZABLE: mọi điểm tranh chấp đều đã được khoá tường minh bằng `FOR UPDATE`, nên không cần trả giá cho việc phải retry hàng loạt do lỗi serialization. Đây cũng là lựa chọn phổ biến trong hệ thống thanh toán thực tế.

4.3. Saga và bù trừ



Hình 7. Sơ đồ phân tích và thiết kế

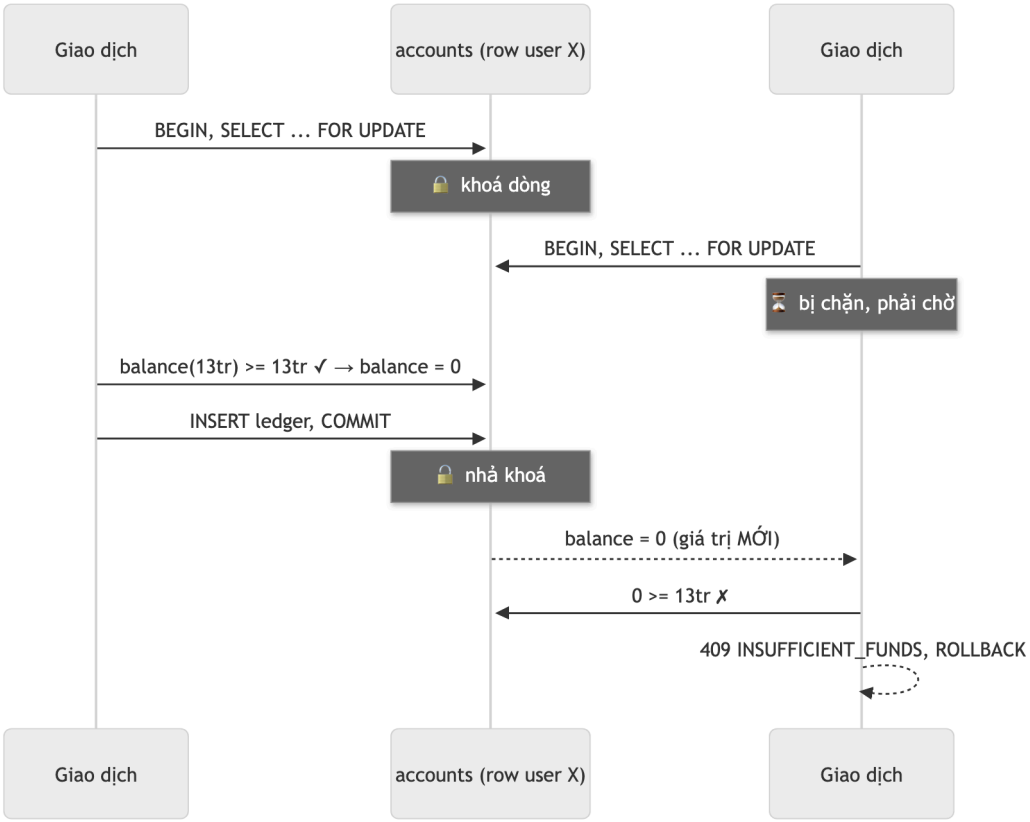
Vì sao trừ tiền trước, gạch nợ sau? Tiền là tài nguyên khan hiếm và **có thể hoàn lại an toàn**; trạng thái học phí thì không nên bị "nhấp nháy" PAID → UNPAID trong mắt sinh viên. Nếu gạch nợ trước rồi trừ tiền lỗi, sẽ có khoảnh khắc học phí hiện "đã đóng" mà chưa ai trả tiền.

Idempotency ở mọi bước: `reference = payment_id` cho lệnh trừ tiền, `payment_ref = payment_id` cho lệnh gạch nợ, `reference = payment_id:refund` cho lệnh hoàn tiền. Retry do timeout mạng không tạo hiệu ứng phụ.

Dấu vết kiểm toán: mỗi bước ghi một dòng vào `payment_events` kèm `request_id`. Giao dịch thất bại có bù trừ để lại chuỗi `DEBIT(OK) → MARK_TUITION_PAID(FAILED) → REFUND(COMPENSATED)` — xem trực tiếp trên giao diện ở mục "Xem các bước xử lý".

4.4. Tình huống 5.1 — Nhiều giao dịch trên cùng một tài khoản

Kịch bản: số dư 13.000.000 ₫, hai giao dịch mỗi giao dịch 13.000.000 ₫ được xác nhận cùng lúc.



Hình 8. Sơ đồ phân tích và thiết kế

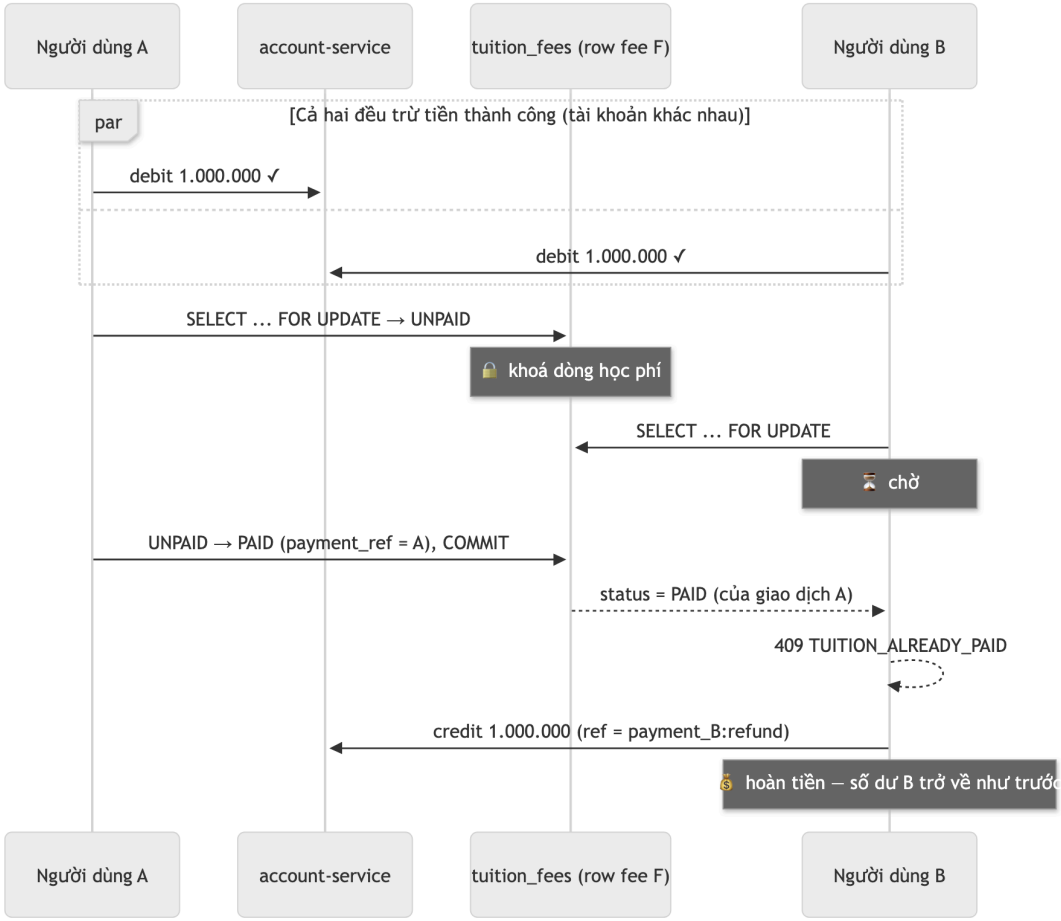
Kết quả bắt buộc: đúng 1 giao dịch `SUCCESS`, 1 giao dịch `FAILED`, số dư cuối `= 0` và **không bao giờ âm**, chỉ 1 khoản học phí được gạch nợ.

Kiểm chứng tự động:

`tests/test_03_concurrency.py::test_51_nhieu_giao_dich_tren_cung_mot_tai_khoan`.

4.5. Tình huống 5.2 — Nhiều tài khoản thanh toán cùng một khoản học phí

Kịch bản: hai người dùng khác nhau cùng đóng học phí cho một MSSV, xác nhận OTP cùng lúc.



Hình 9. Sơ đồ phân tích và thiết kế

Kết quả bắt buộc: khoản học phí `PAID` đúng một lần với `payment_ref` của người thắng; người thua nhận `409 TUITION_ALREADY_PAID`, giao dịch `FAILED` và **số dư không đổi** nhờ bút toán hoàn tiền.

Lớp bảo vệ thứ hai ở `payment-service`: partial unique index `payments(fee_id) WHERE status='SUCCESS'` — dù logic ứng dụng có sai, database vẫn không cho phép hai giao dịch `SUCCESS` trên cùng một khoản học phí.

Kiểm chứng tự động:

`tests/test_03_concurrency.py::test_52_nhieu_tai_khoan_thanh_toan_cung_mot_hoc_phi`.

4.6. Bảng tổng hợp cơ chế

Rủi ro	Cơ chế chính	Cơ chế dự phòng
Chi tiêu vượt số dư	`SELECT ... FOR UPDATE` trên `accounts`	`CHECK balance >= 0`
Trừ tiền hai lần (retry/double-click)	`UNIQUE ledger_entries.reference` + kiểm tra trong khoá	`Idempotency-Key` ở `POST /payments`
Reference dùng lại cho bút toán khác	So khớp tài khoản + chiều + số tiền trước khi coi là replay → `409 REFERENCE_CONFLICT`	`UNIQUE ledger_entries.reference` chặn ghi trùng
Học phí bị thanh toán hai lần	`SELECT ... FOR UPDATE` trên `tuition_fees`	`UNIQUE payments(fee_id) WHERE status='SUCCESS'`, `UNIQUE tuition_fees.payment_ref`
Xác nhận OTP đồng thời cho một giao dịch	OTP dùng một lần (`ACTIVE → USED` trong khoá)	Khoá dòng `payments` + kiểm tra trạng thái `PENDING_OTP`
OTP trùng giữa các giao dịch	`UNIQUE otp_codes.code_hash WHERE status='ACTIVE'`	Sinh lại mã khi đụng ràng buộc
Tiền đã trừ nhưng gạch nợ lỗi	Bù trừ hoàn tiền ngay trong luồng	`payment_events` ghi `COMPENSATED` để đối soát
Brute-force OTP	Đếm `attempts`, khoá sau 5 lần	Mã chỉ sống 5 phút

4.7. Giao dịch treo: job đối soát tự động

Saga chỉ tự bù trừ được **khi tiến trình còn sống**. Nếu `payment-service` bị giết đúng giữa bước trừ tiền và bước gạch nợ, giao dịch nằm lại ở `PROCESSING` với tiền đã trừ — không ai hoàn tác giúp.

`payment-service` chạy một **job đối soát nền** (`services/payment-service/app/reconciliation.py`) mỗi `RECONCILE_INTERVAL_SECONDS` (mặc định 30 giây). Job quét các giao dịch `PROCESSING` có `processing_at` cũ hơn `RECONCILE_STUCK_AFTER_SECONDS` (mặc định 60 giây — saga bình thường chạy dưới 1 giây), rồi **đọc lại sự thật từ hai service sở hữu dữ liệu** để quyết định:

Học phí đã gạch nợ bởi chính giao dịch này?	Có bút toán trừ tiền?	Hành động	Kết quả
có	có	`COMPLETE`	Hoàn tất giao dịch → `SUCCESS`
có	không	`REVERT_AND_FAIL`	Huỷ gạch nợ → `FAILED`
không	có	`REFUND_AND_FAIL`	**Hoàn tiền** → `FAILED`

Học phí đã gạch nợ bởi chính giao dịch này?	Có bút toán trừ tiền?	Hành động	Kết quả
không	không	`FAIL`	Chỉ đóng giao dịch → `FAILED`

Job còn đọc bút toán `payment_id:refund`. Nếu đã hoàn tiền thì không hoàn thêm và không kết luận `SUCCESS`; nếu khoản học phí vẫn thuộc giao dịch đã hoàn tiền, job huỷ gạch nợ đúng reference trước khi chốt `FAILED`. Bút toán phải thuộc đúng người nộp, đúng chiều ghi nợ/ghi có và đúng số tiền. Dữ liệu bất thường hoặc service không truy cập được được giữ lại để thử sau và kiểm tra vận hành.

Các cơ chế bảo vệ:

1. **Idempotent** — hoàn tiền dùng `reference = payment_id:refund`, huỷ gạch nợ dùng `payment_ref = payment_id`.
2. **Khoá xuyên phần xử lý** — foreground commit `PROCESSING` trước khi gọi debit, rồi lấy lại và giữ khóa dòng payment đến khi kết thúc phần thay đổi tiền. Job quét ID, khóa và kiểm tra lại từng dòng bằng `FOR UPDATE SKIP LOCKED`; trạng thái cuối và các event được commit cùng nhau. Helper ghi event chỉ flush, không nhả khóa ngầm.
3. **Không suy đoán từ timeout** — chỉ lỗi nghiệp vụ xác định mới chốt thất bại/bù trừ ngay. Mất phản hồi debit, gạch nợ hoặc hoàn tiền giữ `PROCESSING`: phía nhận có thể đã commit.
4. **Cô lập lỗi mỗi payment** — lỗi dependency/transaction rollback riêng, không ngăn job xử lý các ứng viên tiếp theo.

Nâng cấp database đã có dữ liệu: `Base.metadata.create_all()` chỉ tạo bảng còn thiếu, **không** thêm cột vào bảng đã tồn tại — chạy bản có `processing_at` trên một volume PostgreSQL cũ sẽ lỗi `UndefinedColumn` ở mọi truy vấn payment trong khi `/health` vẫn báo `UP`. `services/payment-service/app/migrations.py` chạy lúc khởi động, **trước** khi bật job đối soát: thêm cột, tạo index và backfill `processing_at = created_at` cho các giao dịch `PROCESSING` cũ. Thiếu backfill thì chúng mang giá trị `NULL`, nằm ngoài mọi so sánh `<` và treo vĩnh viễn ngoài tầm quét. Vì lý do đó bộ lọc quét dùng `coalesce(processing_at, created_at)`: dòng `NULL` do container bản cũ ghi *trong* lúc nâng cấp cuốn chiếu vẫn được đối soát. Các câu lệnh đều idempotent (`ADD COLUMN IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, `UPDATE ... WHERE processing_at IS NULL`) nên chạy lại mỗi lần khởi động vẫn sạch.

Giới hạn: giữ connection database trong lúc gọi HTTP phù hợp tải đồ án nhưng tốn pool khi dịch vụ chậm. Khi triển khai tải lớn cần cơ chế lease/fencing bền vững. Khóa tại payment-service không hủy được RPC đã gửi tới service khác trước khi crash; vì vậy ngưỡng 60 giây

là khoảng chờ vận hành, không phải bảo đảm exactly-once với tác vụ remote trì hoãn vô hạn. Biên lai cho giao dịch được job hoàn tất chưa có outbox/retry tự động; trạng thái và sổ cái mới là nguồn xác nhận thanh toán.

Chạy thủ công một lượt (vận hành hoặc khi demo):

```
curl -s -X POST "http://localhost:8004/internal/payments/reconcile?
stuck_after_seconds=0" \
     -H "X-Internal-Token: tdtu-soa-internal-token" | python3 -m json.tool
```

Kiểm chứng: `tests/test_04_reconciliation.py` dựng trạng thái crash bằng ledger/công nợ thật và ép `PROCESSING` trong database; kiểm tra refund/revert, idempotency, hai worker, khóa dòng và job nền. `tests/test_06_saga_recovery.py` kiểm tra mất phản hồi sau commit và service lỗi rồi phục hồi. Bảng quyết định được kiểm tra riêng ở mức hàm thuần trong `tests/unit/test_reconcile_decisions.py`. `tests/test_07_migration_and_idempotency.py` dựng lại schema cũ trong một schema PostgreSQL riêng rồi chạy đúng các câu lệnh migration thật (backfill, index, chạy lại lần hai), và kiểm tra reference bị dùng lại cho tài khoản / số tiền / chiều khác đều bị từ chối.

4.8. Cách chứng minh khi demo

```
# Chạy riêng bộ test đồng thời và xem kết quả
pytest tests/test_03_concurrency.py -v
```

Ngoài ra có thể quan sát trực tiếp:

1. Đặt `LOCK_HOLD_MS=150` trong `.env` (mặc định) để cửa sổ tranh chấp đủ rộng — khoá giữ lâu hơn, race chắc chắn xảy ra chứ không "may mà đúng".
2. Mở log của service để thấy thứ tự tuần tự hoá:

   ```
   docker compose logs -f account-service tuition-service payment-service
   ```

3. Trên giao diện, mở "Xem các bước xử lý (saga trace)" của giao dịch thất bại để thấy chuỗi `DEBIT → MARK_TUITION_PAID(FAILED) → REFUND(COMPENSATED)`.
4. Đối chiếu tab "Biến động số dư": người thua có đúng một dòng `-1.000.000` và một dòng `+1.000.000`.

> `LOCK_HOLD_MS` chỉ là khoảng `sleep` để *quan sát* tranh chấp; đặt `0` cho môi trường đo hiệu năng. Tính đúng đắn không phụ thuộc vào giá trị này.

5. Hướng dẫn sử dụng và kịch bản demo

5.1. Tài khoản và dữ liệu mẫu

Người dùng iBanking — mật khẩu chung `123456`

Username	Họ tên	SĐT	Email	Số dư ban đầu
`minhnguyen`	Nguyễn Văn Minh	0901234567	minhnguyen@tdtu.edu.vn	50.000.000 ₫
`thuha`	Trần Thu Hà	0912345678	thuha@tdtu.edu.vn	20.000.000 ₫
`quocbao`	Võ Quốc Bảo	0923456789	quocbao@tdtu.edu.vn	15.000.000 ₫
`itdu`	Lê Ít Dư	0934567890	itdu@tdtu.edu.vn	1.000.000 ₫

Sinh viên và học phí — học kỳ `HK1/2026-2027`

MSSV	Họ tên sinh viên	Học phí	Trạng thái	Dùng để demo
`522H0001`	Nguyễn Văn Minh	9.500.000 ₫	UNPAID	Luồng thành công
`522H0002`	Trần Thu Hà	8.750.000 ₫	UNPAID	Thanh toán cho người khác
`522H0003`	Phạm Minh Tuấn	10.200.000 ₫	UNPAID	Dự phòng
`522H0004`	Lê Thị Lan	7.300.000 ₫	**PAID**	Học phí đã đóng
`522H0005`	Võ Quốc Bảo	11.000.000 ₫	UNPAID	Hai người cùng đóng

5.2. Hướng dẫn sử dụng hệ thống

Bước 1 — Đăng nhập

Mở http://localhost:3000, nhập username/password. Sau khi đăng nhập, thanh trên cùng hiển thị họ tên và **số dư khả dụng**.

Bước 2 — Thông tin người nộp tiền

Khối **(a) Người nộp tiền** tự động điền họ tên, số điện thoại, email từ tài khoản đang đăng nhập và **không cho chỉnh sửa** — đúng yêu cầu đề bài.

Bước 3 — Tra cứu học phí

Nhập MSSV vào khối **(b) Thông tin học phí** rồi bấm **Tra cứu** (hoặc Enter). Hệ thống hiển thị họ tên sinh viên, ngành/khoa, học kỳ và **số tiền học phí còn phải thanh toán**. Nếu MSSV không tồn tại → thông báo lỗi; nếu đã đóng đủ → cảnh báo "không còn khoản học phí cần thanh toán".

Bước 4 — Xác nhận giao dịch

Khối **(c) Thông tin thanh toán** hiển thị số dư khả dụng, số tiền cần thanh toán và số dư dự kiến sau giao dịch. Tick **đồng ý điều khoản** (bấm vào liên kết để đọc). Nút **Xác nhận giao dịch** chỉ bật khi: 1. đã tra cứu được khoản học phí **chưa thanh toán**; 2. **số dư ≥ số tiền** cần thanh toán; 3. đã đồng ý điều khoản.

Dòng gợi ý dưới nút luôn cho biết còn thiếu điều kiện nào.

Bước 5 — Nhập OTP

Hệ thống gửi **mã OTP 6 chữ số** tới email người nộp tiền và mở hộp thoại xác thực kèm **đồng hồ đếm ngược 5 phút**. Mở **MailHog** tại http://localhost:8025 để đọc email và lấy mã. Có thể bấm **Gửi lại mã** — mã cũ lập tức mất hiệu lực.

Bước 6 — Kết quả

Xác thực đúng → hộp thoại **Giao dịch thành công** hiển thị mã giao dịch, số tiền, số dư còn lại, kèm mục **"Xem các bước xử lý (saga trace)"**. Email biên lai được gửi ngay sau đó (xem trong MailHog).

Bước 7 — Lịch sử

Tab **Lịch sử giao dịch** gồm hai bảng: danh sách giao dịch học phí (kèm trạng thái) và biến động số dư (ghi nợ/ghi có). Bấm **Chi tiết** để xem lại saga trace của bất kỳ giao dịch nào.

5.3. Kịch bản demo trước hội đồng (≈ 10 phút)

Màn 1 — Kiến trúc (1 phút)

```
curl -s http://localhost:8080/health/services | python3 -m json.tool
docker compose ps
```

Chỉ ra 11 container đang chạy: 6 microservice, gateway, PostgreSQL (6 database), MailHog, web và Jaeger; đối chiếu sơ đồ ở 02-kien-truc-microservices.md.

Màn 2 — Luồng thành công (2 phút)

Đăng nhập `minhnguyen` → tra cứu `522H0001` → xác nhận → lấy OTP trong MailHog → hoàn tất. Nhấn mạnh: số dư giảm đúng 9.500.000 ₫, học phí chuyển **PAID**, lịch sử có bản ghi, email biên lai đã tới.

Màn 3 — Thanh toán cho sinh viên khác (30 giây)

Vẫn tài khoản `minhnguyen`, tra cứu `522H0002` → thanh toán hộ. Chứng minh yêu cầu "thanh toán cho chính mình hoặc sinh viên khác".

Màn 4 — Các ràng buộc OTP (1,5 phút)

- Nhập sai mã → `Mã OTP không đúng` kèm số lần thử còn lại.
- Bấm **Gửi lại mã**, dùng mã cũ → bị từ chối.
- Tra cứu `522H0004` (đã đóng) → hệ thống chặn ngay từ bước tra cứu.

Màn 5 — Concurrency 5.1 và 5.2 (3 phút) ⭐

```
pytest tests/test_03_concurrency.py -v
```

Vừa chạy vừa giải thích: - `test_51...`: hai giao dịch trên cùng tài khoản, số dư chỉ đủ một → 1 SUCCESS, 1 `INSUFFICIENT_FUNDS`, số dư cuối bằng 0. - `test_52...`: hai người cùng đóng một khoản học phí → 1 SUCCESS, 1 `TUITION_ALREADY_PAID` **và được hoàn tiền**. - `test_xac_nhan_trung_lap...`: bấm xác nhận 3 lần cùng lúc → chỉ trừ tiền một lần.

Sau đó mở giao diện, xem **saga trace** của giao dịch thất bại để chỉ ra chuỗi `DEBIT → MARK_TUITION_PAID(FAILED) → REFUND(COMPENSATED)` và dòng hoàn tiền trong bảng biến động số dư.

Làm thủ công nếu muốn: mở hai trình duyệt (một cửa sổ ẩn danh), đăng nhập `minhnguyen` và `thuha`, cả hai cùng tra cứu `522H0005`, cùng tạo giao dịch, lấy hai mã OTP trong MailHog, rồi bấm **Xác nhận** gần như đồng thời.

Màn 6 — Toàn bộ test tự động (1 phút)

```
pytest -v
```

Màn 7 — Unit test và test pyramid

```
pytest tests/unit -v
pytest -m unit -v
```

Unit test kiểm tra logic thuần (OTP, JWT, email, quyết định đối soát) và tracing in-memory, không cần Docker. Integration test kiểm tra HTTP/PostgreSQL và lỗi giữa saga; E2E đi từ tạo payment qua OTP/email đến số dư/công nợ/biên lai. Concurrency test thuộc tầng integration, không phải một tầng riêng của pyramid. Mục tiêu là nhiều phản hồi nhanh ở tầng unit, giữ test xuyên dịch vụ cho những tính chất không thể chứng minh bằng mock.

Màn 8 — Jaeger và phục hồi giao dịch treo

1. Mở http://localhost:16686, chọn service `api-gateway`, thực hiện một luồng thanh toán mới và tìm trace theo thời gian.
2. Mở waterfall: gateway → payment → account/tuition/otp/notification, kiểm tra cùng trace ID và quan hệ parent-child.
3. Chạy `pytest tests/test_04_reconciliation.py tests/test_06_saga_recovery.py -v` để kiểm tra trạng thái sau crash và mất phản hồi sau commit; test nền chờ job tự xử lý thay vì chỉ gọi API thủ công. `pytest tests/test_05_tracing.py -v` kiểm tra tự động trace xuyên service.
4. Đối chiếu balance, ledger, trạng thái fee/payment và span `payment.reconcile`.

Màn 9 — Tài liệu API (1 phút)

Mở http://localhost:8004/docs (payment-service) — Swagger sinh tự động, có request/response model, mã lỗi và mô tả từng endpoint.

5.4. Câu hỏi thường gặp khi bảo vệ

Hỏi: Vì sao không dùng một transaction duy nhất cho cả hệ thống? Mỗi service có database riêng nên không thể dùng một transaction ACID xuyên service. Hệ thống dùng saga có bù trừ: mỗi bước là một transaction cục bộ ACID, nếu bước sau hỏng thì hoàn tác bước trước bằng nghiệp vụ nghịch đảo (hoàn tiền). Xem 04-transaction-concurrency.md.

Hỏi: Nếu hệ thống sập ngay sau khi trừ tiền thì sao? Job đối soát **đã được hiện thực** trong payment-service: mặc định chạy mỗi 30 giây, xét `PROCESSING` cũ hơn 60 giây. Job đọc công nợ và cả bút toán debit/refund: hoàn tất nếu đã gạch nợ và chưa hoàn tiền; hoàn tiền nếu chỉ mới debit; huỷ gạch nợ nếu không còn khoản tiền tương ứng. Mỗi payment được khóa riêng để không tranh chấp với foreground/worker khác. Nếu dependency chưa phục hồi, job giữ trạng thái để thử lại. Xem mục 4.7 về giới hạn RPC đang bay và biên lai best-effort.

Hỏi: OTP có an toàn không? Mã không lưu dạng plaintext (SHA-256 kèm pepper), gắn cứng với `payment_id`, có partial unique index đảm bảo không trùng mã giữa các giao dịch còn hiệu lực, hết hạn sau 5 phút, chỉ dùng được một lần, và bị khoá sau 5 lần nhập sai.

Hỏi: Tại sao trừ tiền trước rồi mới gạch nợ? Debit là thao tác có thể bù trừ bằng credit cùng reference; đặt nó trước giúp tránh gạch nợ khi tài khoản không đủ tiền. Nếu kết quả remote chưa rõ thì giữ `PROCESSING` và đối soát, không khẳng định mọi lỗi đều được xử lý tức thời; xem giới hạn ở mục 4.7.

Hỏi: Đây có thực sự là microservices không, hay chỉ là monolith chia thư mục? Mỗi service là một container riêng, một database riêng, một tiến trình riêng, chỉ giao tiếp qua REST; có thể tắt một service và thấy đúng service phụ thuộc trả `503 SERVICE_UNAVAILABLE`:

```
docker compose stop tuition-service
curl -i http://localhost:8080/api/v1/tuitions/522H0001 -H "Authorization: Bearer
<token>"
docker compose start tuition-service
```

6. Kiểm thử và minh chứng thực nghiệm

6.1. Môi trường và kết quả

Ngày kiểm chứng: 14/09/2026. Backend chạy Python 3.11 trong Docker Compose, PostgreSQL 16, Jaeger 2.20.0; bộ kiểm thử chạy bằng Python 3.11.15 và pytest 8.3.4.

Lệnh chạy: `python3.11 -m pytest -q --junitxml=docs/report-assets/test-results.xml` trong môi trường đã cài `tests/requirements.txt`.

Kết quả toàn bộ: 88 passed, 1 skipped trong 53,25 giây. Một cảnh báo deprecation của Starlette/AnyIO, không phải lỗi nghiệp vụ. File JUnit `report-assets/test-results.xml` lưu kết quả từng test. Test skipped kiểm tra OTP hết hạn, tách riêng để không phải đợi 300 giây trong mỗi lần chạy. Lượt riêng với `OTP_TTL_SECONDS=8` đạt **1 passed trong 10,46 giây** (`report-assets/test-otp-expiry.xml`); sau đó đã khôi phục và kiểm tra TTL **300 giây** trong container. Như vậy tất cả 89 ca đã được thực thi thành công qua hai cấu hình, không phải 89 ca pass trong một lượt chạy.

Nhóm	Số ca	Kiểm chứng
Unit	42	JWT, OTP, email, bảng đối soát, span/log và HTTPX sync/async
Payment integration/E2E	11	Phân quyền, công nợ, idempotency, số dư, biên lai
OTP integration	7	Sáu ca pass ở TTL 300 giây; ca hết hạn chạy riêng
Concurrency	3	Hai tình huống đề bài, xác nhận trùng
Đối soát	15	Crash state, refund/revert, worker đồng thời, khóa, job nền
Jaeger	1	Trace qua sáu service và parent-child
Saga recovery	5	Mất phản hồi sau commit, dependency lỗi rồi phục hồi
Nâng cấp schema và idempotency sổ cái	5	Migration `processing_at` trên database cũ; reference dùng lại sai tài khoản/số tiền/chiều

Tách unit khỏi Docker: cả `pytest tests/unit` và `pytest -m unit` chạy được với `GATEWAY_URL=http://127.0.0.1:1`. Test pyramid phân biệt theo phạm vi và tốc độ, không coi số lượng ca là chứng minh đầy đủ chất lượng. Database và khóa trong các test

concurrency/recovery là PostgreSQL thật; chỉ adapter HTTP được chèn lỗi sau thao tác remote đã commit.

6.2. Truy vết phân tán

Trace minh chứng: `9515ba3ab0b54e8cbb878d91214fb7e1`, request tạo giao dịch qua gateway. Kiểm thử xác nhận cùng trace ID ở gateway, payment, account, tuition, otp và notification; cạnh parent-child nối đúng các lời gọi. JSON access log của gateway/payment/OTP khớp ID này. Đây là một mẫu quan sát, không phải benchmark hiệu năng.

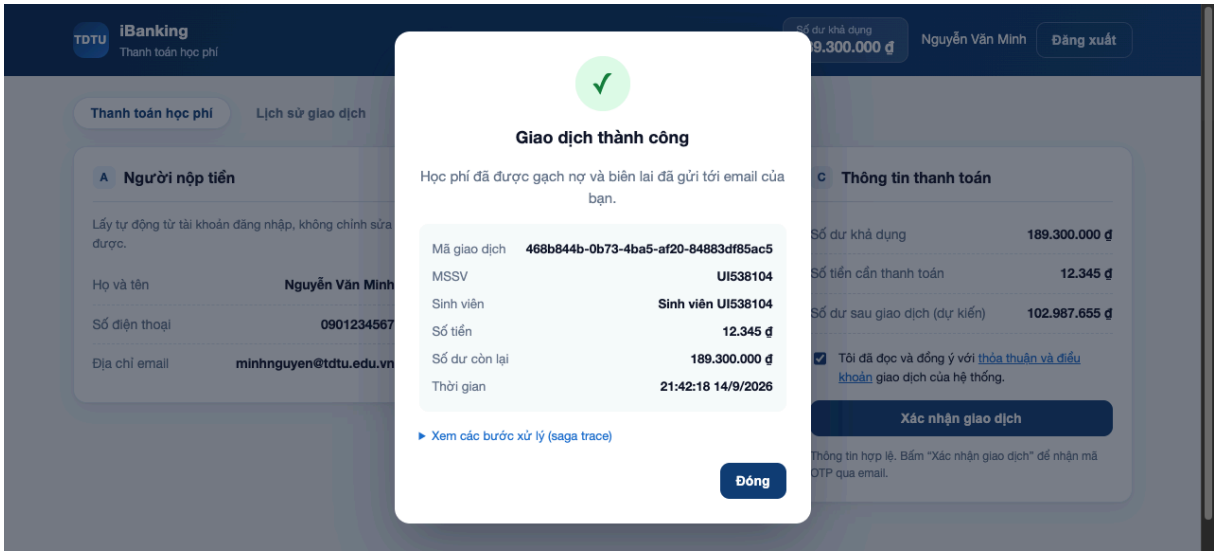


Waterfall Jaeger của request tạo payment qua sáu service

Jaeger dùng bộ nhớ nên trace ID không còn tra cứu được sau khi container restart. Ảnh và kết quả test lưu lại làm minh chứng.

6.3. Giao diện và luồng sử dụng

Kiểm tra trên trình duyệt với khoản học phí riêng `UI538104`: đăng nhập, tra cứu, đồng ý điều khoản, tạo giao dịch, lấy OTP MailHog, xác nhận 12.345 đồng, xem lịch sử và kiểm tra không thể đóng lại cùng khoản nợ. Mã thanh toán: `468b844b-0b73-4ba5-af20-84883df85ac5`. Không có lỗi JavaScript được ghi nhận trong luồng này.



Giao dịch thành công trên giao diện iBanking

6.4. Phạm vi chứng minh

Mô phỏng crash dùng trạng thái ledger/công nợ/payment thật tại các điểm gián đoạn; không phải thử nghiệm cắt điện máy chủ. Test response-loss gọi thao tác remote thật rồi ném lỗi thay cho phản hồi. Các ca chứng minh cơ chế ở phạm vi kiểm thử, không chứng minh exactly-once dưới mọi phân hoạch mạng hoặc chịu tải production. Unit test chưa đo coverage theo dòng/nhánh; tỷ lệ pass không đồng nghĩa 100% coverage.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được

Hệ thống mô phỏng luồng thanh toán học phí và các yêu cầu transaction/concurrency của đề bài:

1. **Microservices**: Sáu service nghiệp vụ, API Gateway và web frontend; mỗi service nghiệp vụ sở hữu một database trong PostgreSQL và trao đổi qua REST.
2. **Saga và phục hồi**: `payment-service` điều phối OTP, debit, gạch nợ, hoàn tiền và lưu trạng thái. Job nền đọc lại ledger/công nợ để xử lý `PROCESSING` sau gián đoạn, không suy đoán kết quả remote từ timeout. Migration lúc khởi động thêm và backfill `processing_at`, nên nâng cấp trên database đã có dữ liệu không làm mất dấu các giao dịch đang treo.
3. **Concurrency**: Khóa dòng PostgreSQL `SELECT ... FOR UPDATE`, `SKIP LOCKED`, CHECK/UNIQUE constraint và idempotency theo reference. Vì `reference` là UNIQUE toàn hệ thống, replay chỉ được công nhận khi trùng cả tài khoản, chiều và số tiền; lệch thì trả `409 REFERENCE_CONFLICT` thay vì báo hoàn tiền thành công sai. Hệ thống không dùng Redis, Redlock hay khóa phân tán trong bộ nhớ.
4. **Bảo mật và truy vết**: JWT tại gateway và các service, token nội bộ, OTP băm SHA-256 với pepper và hạn dùng 5 phút. OpenTelemetry FastAPI/HTTPX xuất span sang Jaeger; correlation ID cùng trace ID/span ID hỗ trợ truy tìm qua log.

Các giới hạn vận hành được nêu ở chương 4: nhất quán cuối cùng, connection giữ trong phần saga, remote RPC chưa có fencing, Jaeger lưu trong bộ nhớ và biên lai chưa có outbox. Đây là mô hình đồ án, không phải chứng nhận hệ thống ngân hàng production.

Hướng phát triển

- Tích hợp Message Broker (Apache Kafka / RabbitMQ) để chuyển đổi từ HTTP Saga sang Event-Driven Saga cho các tác vụ bất đồng bộ quy mô lớn.
- Bổ sung Outbox Pattern kết hợp Debezium CDC để giải quyết triệt để dual-write giữa database và message broker.
- Mở rộng phân hệ đối soát tự động hàng ngày (End-of-day Reconciliation) đối chiếu sổ cái giữa Ngân hàng và Trường đại học.

TÀI LIỆU THAM KHẢO

Các tài liệu kỹ thuật được đối chiếu ngày 14/09/2026:

1. PostgreSQL Global Development Group. *PostgreSQL 16 — Explicit Locking*: khóa dòng, `SELECT FOR UPDATE` và thời điểm giải phóng khóa. https://www.postgresql.org/docs/16/explicit-locking.html.
2. OpenTelemetry. *Manual instrumentation for OpenTelemetry Python*: `TracerProvider`, span, processor và exporter. https://opentelemetry.io/docs/languages/python/instrumentation/.
3. Chris Richardson. *Pattern: Saga*: local transaction, orchestration và compensating transaction. https://microservices.io/patterns/data/saga.html.

Minh chứng của hiện thực nằm trong mã nguồn, bộ kiểm thử `tests/` và các file JUnit/ảnh tại `docs/report-assets/`; tài liệu tham khảo mô tả mẫu thiết kế, không thay thế kiểm chứng hệ thống.